cantera /

TORNEO INFANTIL DE FÚTBOL

PROYECTO FINAL · DESARROLLO WEB

El torneo, en tus manos.

Manual técnico, guía de uso
y preparación para publicación.

Symfony 7.4 · Doctrine · Twig · MySQL 8.4

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información
Centro Universitario Villanueva · Jornada sábado

Documento actualizado: 16 de octubre de 2026
Entrega indicada en el enunciado: 16 de octubre de 2026
Presentación indicada: 17 de octubre de 2026

Los nombres de los integrantes y sus comentarios personales deberán ser incorporados por el grupo. Se utiliza un enlace temporal de Cloudflare para presentar; el alojamiento permanente sigue pendiente.

01 / ALCANCE

Una aplicación para organizar el torneo

Cantera permite administrar los datos de jugadores, equipos, jornadas, goles e incidencias de un torneo infantil. La implementación amplía los CRUD originales sin reemplazar sus entidades ni perder los registros existentes.

Requisito del proyecto	Dónde se resuelve
Datos personales, fotografía y equipo	Jugadores → Inscribir jugador / Editar / Ver ficha
Catálogo de nombres de equipos	Equipos → Registrar equipo
Catálogo de jornadas y fechas	Jornadas → Crear jornada
Goles por jugador y jornada	Goles → Registrar goles
Tarjetas, descripción y suspensión	Incidencias → Registrar incidencia
Lista para el árbitro con fotos y suspensión	Reportes → Nómina del árbitro
Incidencias por equipo y jugador	Reportes → Informe de incidencias
Goleadores de mayor a menor	Reportes → Goleadores
Consulta completa de jugador	Jugadores → Ver ficha
Datos precargados, documento y fuentes	Demo, este manual y paquete ZIP
Publicación y URL	Enlace temporal de Cloudflare; alojamiento permanente pendiente

Reglas explícitas

- La suspensión de una tarjeta roja corresponde a una fecha exacta. El árbitro ve “Suspendido” cuando la fecha de la jornada coincide con ella.
- La tarjeta amarilla no genera suspensión automática. El enunciado no define acumulación de amarillas ni sanciones por varios partidos.
- Hay un único total de goles por jugador/jornada. Si cambia el resultado, se edita el registro existente.
- Las fotografías son opcionales en el registro, para mantener la compatibilidad con los datos anteriores. La ficha muestra iniciales si no hay imagen cargada.

> Los registros Demo son ficticios. Sus imágenes son avatares ilustrativos originales; no representan fotografías de menores reales.

Preparar y ejecutar el proyecto

Prerrequisitos

PHP 8.2 o posterior compatible con Symfony 7.4, Composer 2, navegador moderno y acceso a una terminal. Se verificó con PHP 8.2.12 y Chrome. XAMPP puede proporcionar PHP. El servidor utilizado es MySQL Community Server 8.4; MariaDB de XAMPP no sustituye este requisito.

Extensiones necesarias para este proyecto: `ctype, iconv, pdo_mysql, fileinfo, mbstring; pdo_sqlite para importar el respaldo y ejecutar las pruebas en memoria` según las dependencias instaladas, además de DOM/XML para pruebas. Composer comprueba los requisitos reales del archivo lock. No uses `--ignore-platform-reqs` para ocultar extensiones faltantes. La aplicación no necesita GD para la validación configurada.

```
php -v
php -m
composer --version
composer install
composer check-platform-reqs
```

Base de datos MySQL

Crea una base MySQL `cantera_torneo` y un usuario con permisos. Configura la conexión real en `.env.local`:

```
DATABASE_URL="mysql://USUARIO:CLAVE@127.0.0.1:3307/cantera_torneo?serverVersion=8.4.9&charset=utf8mb4"
```

`.env.local`, si existe, tiene prioridad y debe conservarse. Ejecuta las migraciones para crear las tablas o aplicar cambios pendientes:

```
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console importmap:install
php bin/console app:demo
```

El paquete incluye un SQL de demostración y el comando recomendado para cargar los datos. El comando `app:demo` no duplica los datos si el equipo marcador sigue presente. En una instalación vacía crea cuatro equipos, doce jugadores, cuatro jornadas, doce registros de goles y cuatro incidencias.

Iniciar en Windows / PowerShell

Primero asegúrate de que MySQL esté activo. En esta computadora se configuró una instancia independiente; consulta `docs/Manual-Cantera.pdf` para iniciarla después de reiniciar Windows.

```
cd C:\Users\dayli\ProyectoFinalA
php -S 127.0.0.1:8000 -t public bin/router.php
```

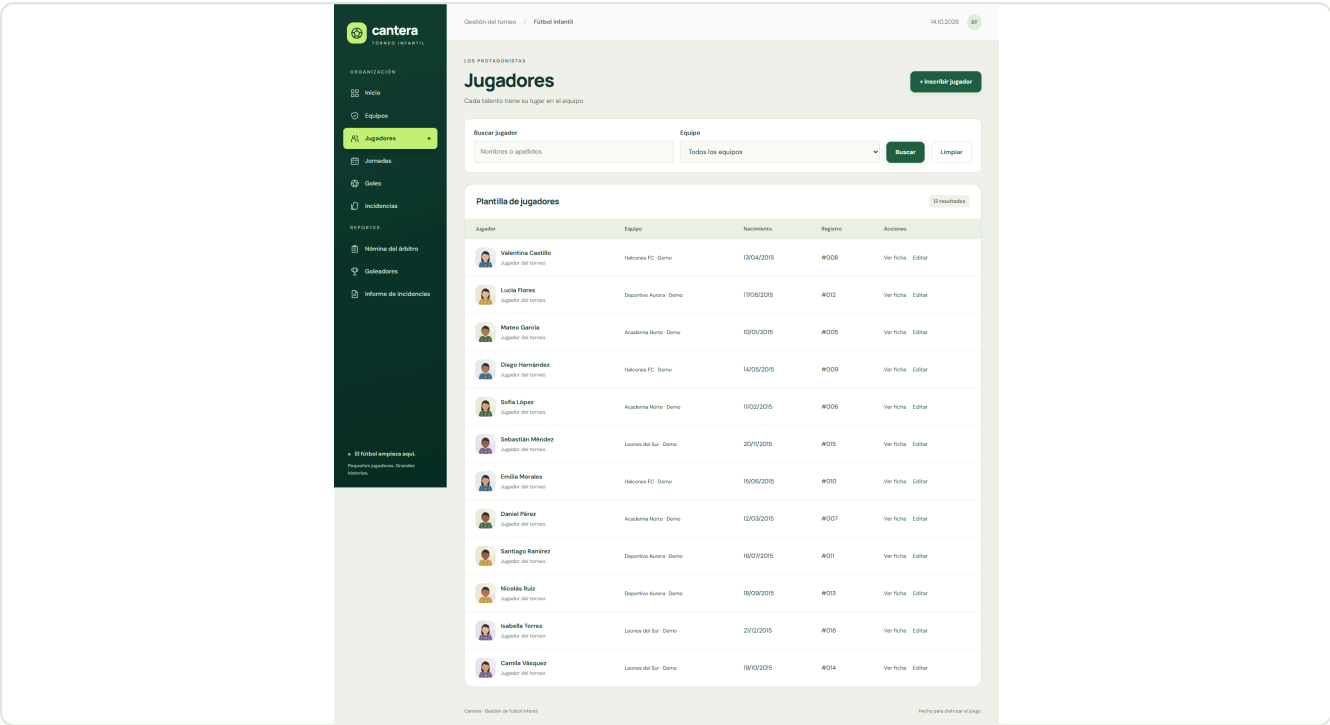
Abre **http://127.0.0.1:8000**. Mantén abierta la terminal y usa Ctrl+C para detener el servidor. El router de desarrollo permite que Symfony entregue sus assets. También puede utilizarse Symfony CLI si está instalado.

> No importes datos Demo sobre una base que contiene registros existentes. La copia del proyecto original se encuentra en `var/backups/` en el equipo de trabajo. Los respaldos privados no se incluyen en el ZIP de entrega.

03 / USO DIARIO

Registrar equipos y jugadores

1. Abre **Equipos** y pulsa **Registrar equipo**. El nombre es obligatorio.
2. Abre **Jugadores** → **Inscribir jugador**. Completa nombres y apellidos en campos separados, fecha de nacimiento y equipo.
3. Selecciona una fotografía JPEG, PNG o WebP de hasta 2 MB. El navegador presenta una vista previa; el servidor vuelve a validar el archivo.
4. Guarda. El jugador aparece en la lista con su equipo y fotografía. Puedes buscar por nombres/apellidos o filtrar por equipo.
5. En **Ver ficha** se muestran datos personales, fotografía, goles por jornada e incidencias con sus fechas de suspensión.



Editar y eliminar

Editar sin seleccionar un archivo conserva la fotografía actual. Subir otra genera un nuevo nombre y mantiene la imagen anterior en disco. Los valores antiguos del campo fotografía permanecen como texto en la ficha y en edición, sin usarlos como rutas inseguras.

La eliminación pide confirmación en el navegador y verifica CSRF en el servidor. Un equipo con jugadores no se elimina. Un jugador con goles o incidencias conserva su historial hasta resolver los registros dependientes.

Datos inválidos

Nombres y apellidos son obligatorios, de hasta 100 caracteres: admiten letras, tildes, ñ, espacios, guiones y apóstrofes; rechazan números y otros símbolos. El jugador debe tener al menos 6 años cumplidos al crear o editar, sin edad máxima configurada. Se rechazan fechas vacías, inexistentes, futuras y edades menores de 6 años. El equipo debe existir. Las fotografías admiten JPEG, PNG o WebP, hasta 2 MB y 4096 × 4096 píxeles. Las reglas se aplican en el servidor aunque se omita la validación HTML. Los registros antiguos no se corrigen automáticamente.

Jornadas, goles e incidencias

Preparar las jornadas al inicio

Desde **Jornadas** → **Crear jornada** registra un nombre y la fecha de juego. Estas jornadas aparecen en los formularios de goles y en los reportes. Se pueden editar. Una jornada con goles registrados no se elimina.

Registrar goles

1. Selecciona **Goles** → **Registrar goles**.
2. Elige jugador y jornada; ingresa una cantidad entera de 1 a 100.
3. Guarda el total. Si ese jugador ya tiene registro en la jornada, el formulario indica que debe editarse el registro existente.

La validación de Symfony previene duplicados y la base de datos también contiene un índice único sobre `jugador_id + jornada_id`.

Registrar tarjetas

En **Incidencias** selecciona al jugador, escribe la descripción y elige tarjeta roja o amarilla. Indica la fecha de la incidencia. La tarjeta roja requiere fecha de suspensión; esta fecha no puede ser anterior a la incidencia. La amarilla no permite fecha de suspensión.

Ejemplo	Resultado esperado
Roja del 10/10, suspensión 17/10	Suspensión válida
Roja sin suspensión	Mensaje de validación
Amarilla con suspensión	Mensaje de validación
Suspensión 09/10, incidencia 10/10	Mensaje de validación
Jornada 17/10 para el primer caso	Jugador suspendido
Jornada 24/10 para el primer caso	Jugador habilitado

Corregir un registro

Las listas de goles e incidencias incluyen **Editar**. Desde ese formulario también puede eliminarse el registro con confirmación. Las operaciones de escritura utilizan POST y protección CSRF.

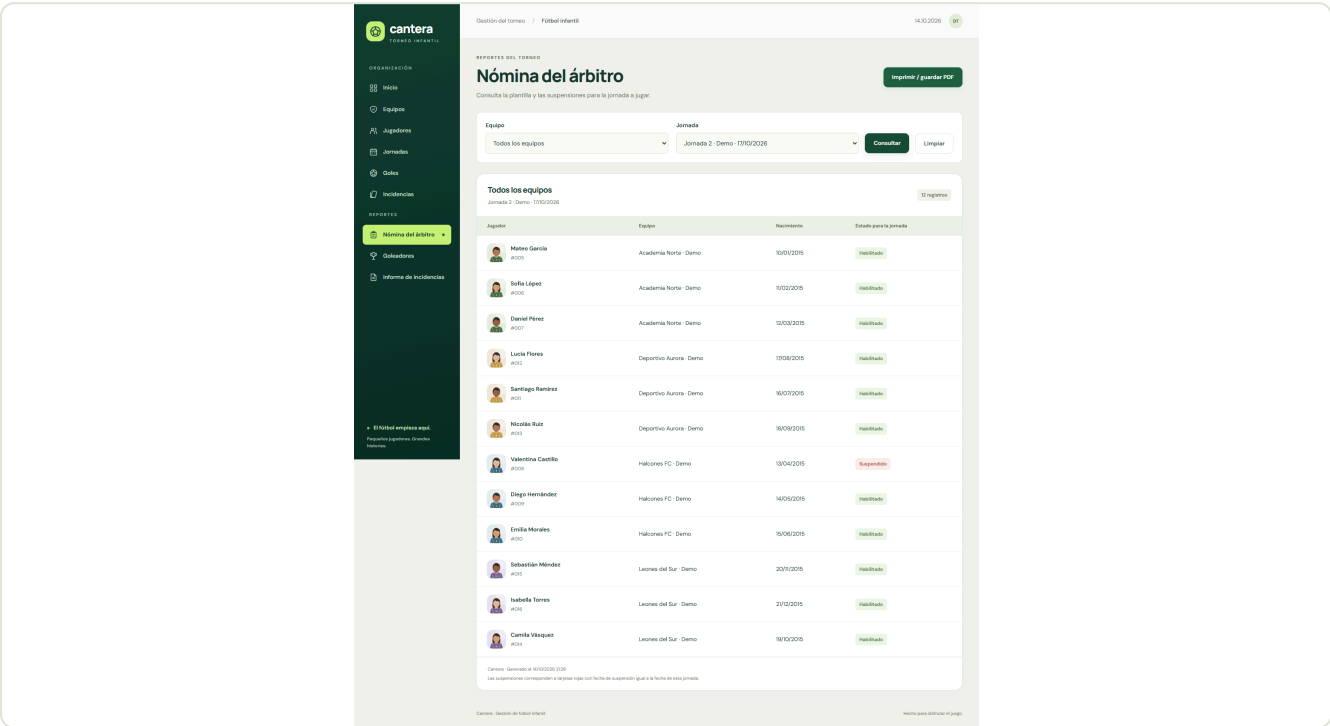
> El sistema registra los datos que ingresa el organizador. No implementa partidos, marcadores por equipo, posiciones de equipos ni sanciones acumulativas porque no forman parte de los requisitos del documento.

05 / REPORTES

Preparar la información del torneo

Nómina del árbitro

Selecciona la jornada a jugar y, opcionalmente, un equipo. El reporte incluye nombres, fotografía, equipo, fecha de nacimiento y estado de habilitación. La consulta compara la fecha de suspensión de las tarjetas rojas con la fecha de la jornada seleccionada.



Informe de incidencias

Puedes consultar todos los equipos, uno específico y uno o todos sus jugadores. Si eliges un jugador de otro equipo, el formulario muestra un error en lugar de presentar una consulta incoherente.

Tabla de goleadores

Se totalizan los goles de cada jugador con `SUM(g.cantidad)`. El orden es descendente y muestra fotografía, nombres y equipo. Los jugadores sin goles no aparecen en la tabla. Los empates usan orden alfabético y luego el identificador para un resultado estable.

Imprimir y guardar como PDF

Después de consultar, pulsa **Imprimir / guardar PDF**. En el diálogo del navegador elige impresora o Guardar como PDF. El diseño de impresión utiliza A4 horizontal, oculta navegación y filtros, y repite las cabeceras de tabla cuando hay varias páginas.

El reporte de ejemplo antiguo se archivó; genera uno actualizado desde la página. Los reportes siempre se calculan con los datos actuales de la base.

06 / ARQUITECTURA

Entender las piezas de Symfony

Frontend: frontend/templates contiene las pantallas Twig y frontend/assets contiene CSS y JavaScript. **Backend:** backend/src contiene la lógica PHP, formularios, consultas y seguridad. La configuración compartida permanece en config/. Es una sola aplicación Symfony que genera HTML en el servidor. Docker no se usa y su configuración fue retirada. Los ZIP antiguos se archivaron en var/backups/entregas-anteriores; entrega/ conserva el paquete vigente.

El proyecto sigue el patrón modelo–vista–controlador. Doctrine convierte las entidades PHP en filas de MySQL. Twig presenta los datos. Los formularios y el validador comprueban las solicitudes antes de guardarlas.

Capa	Archivos principales	Responsabilidad
Entidades	backend/src/Entity	Datos, relaciones y reglas de validación
Formularios	backend/src/Form	Campos, etiquetas y selección de entidades
Controladores	backend/src/Controller	Rutas, lectura del formulario y respuestas
Servicios	EstadisticasTorneo, FotografiaUploader	Consultas de reportes y archivos
Vistas	frontend/templates	HTML reutilizable con Twig
Presentación	frontend/assets/styles/app.css, frontend/assets/app.js	Diseño adaptable, menú y vista previa
Migraciones	migrations	Versiones del esquema de datos
Pruebas	tests/Controller	Comportamiento completo de los formularios y reportes

Relaciones

```
Equipo 1 ——— N Jugador
Jugador 1 ——— N Gol N ——— 1 Jornada
Jugador 1 ——— N Incidencia
```

Las relaciones originales `Equipo.jugadors` y `Jugador.equipo` se conservaron. No se renombraron las tablas originales. Las nuevas entidades son Jornada, Gol e Incidencia. Las migraciones activas para MySQL están en `migrations/mysql/`; las originales SQLite permanecen conservadas, pero no se ejecutan.

Flujo de una solicitud

```
GET /gol/new → formulario Twig
POST /gol/new → handleRequest() → isSubmitted() + isValid()
→ persist() + flush() → redirección 303 → lista actualizada
```

`RegistroController` comparte ese flujo para los tres módulos nuevos utilizando una lista cerrada de entidades y formularios. El usuario no puede suministrar arbitrariamente un nombre de clase. Los controladores originales de equipos y jugadores se conservaron y ampliaron.

Las consultas de reportes se concentran en `EstadisticasTorneo` para evitar repetir fórmulas en los controladores y permitir pruebas directas de los resultados.

Cómo se construyeron los cambios

1. Se revisaron las entidades, formularios, rutas y plantillas existentes. Se respaldaron las fuentes y la base original antes de ampliar y migrar a MySQL.
2. Se añadieron atributos Doctrine a las nuevas entidades y restricciones del validador. Una validación Callback resuelve las reglas condicionales de suspensión.
3. Se generó una migración inicial específica para MySQL y se copió el contenido del respaldo SQLite sin modificarlo. Se comprobó que el esquema coincide con las entidades.
4. Se construyeron formularios con EntityType, DateType, IntegerType y ChoiceType, reutilizando la protección CSRF de Symfony.
5. Se añadieron las consultas agregadas y los filtros de reportes. Las entradas se pasan como parámetros, no mediante concatenación de SQL.
6. Se integraron las vistas Twig, estilos adaptables, navegación móvil, estados vacíos y mensajes de confirmación.
7. Se cargaron datos ficticios mediante un comando repetible y se verificaron formularios y reportes en pruebas aisladas.

Carga de fotografías

`fotografiaArchivo` utiliza `mapped: false`. Es un archivo temporal y no se asigna directamente a la propiedad de texto `fotografia`. Después de validar, un servicio genera un nombre con `random_bytes(16)`, determina la extensión por el MIME y mueve el archivo fuera de la carpeta pública.

La ruta `/jugador/{id}/fotografia` entrega únicamente nombres que cumplen el patrón seguro. Incluye `X-Content-Type-Options: nosniff`. No se aceptan SVG ni archivos ejecutables. La implementación comprueba tipo, tamaño y dimensiones; no recodifica la imagen ni elimina metadatos.

Conservar los datos

La foto actual no se borra cuando el formulario no incluye otra. Las imágenes anteriores permanecen en disco al reemplazarse. Un fallo de escritura impide guardar el jugador. Si falla MySQL después de guardar el archivo, puede quedar una imagen sin referencia; la limpieza debe hacerse mediante una tarea explícita y con respaldo.

Recomendaciones de desarrollo

Usa migraciones para cambiar el esquema, mantén secretos en variables de entorno, ejecuta las pruebas antes de publicar, separa las reglas de negocio de Twig y evita borrar dependencias en cascada sin una decisión consciente. Para modificar una regla, actualiza su validación y su prueba correspondiente.

Pasar del equipo local al hosting

Para mantener el sitio disponible con la computadora apagada falta elegir un alojamiento permanente. Necesitas un servidor compatible con PHP 8.2+, Composer y MySQL 8.4, con almacenamiento persistente y posibilidad de apuntar la raíz web a `public/`. Un hosting solo para HTML estático no ejecuta Symfony.

1. Copia las fuentes y configura un dominio con HTTPS. El único directorio expuesto debe ser `public/`; nunca la raíz del repositorio.
2. Crea un `.env.local` exclusivo del servidor o configura variables de entorno equivalentes. Genera un APP_SECRET nuevo; no reutilices el de otro despliegue.
3. Instala dependencias de producción, aplica migraciones, instala el importmap y compila assets.
4. Permite escritura al usuario del servidor en `var/cache`, `var/log` y `var/uploads/jugadores`.
5. Configura Nginx o Apache para enviar rutas que no existen a `public/index.php`. Usa PHP-FPM o el mecanismo PHP de tu proveedor.
6. Verifica páginas, imágenes, formularios, reportes y registros de errores antes de compartir la URL.

```
APP_ENV=prod
APP_DEBUG=0
APP_SECRET=GENERAR_UN_VALOR_ALEATORIO_NUEVO
DATABASE_URL="mysql://USUARIO:CLAVE@127.0.0.1:3307/cantera_torneo?serverVersion=8.4.9&charset=utf8mb4"
```

```
composer install --no-dev --optimize-autoloader
php bin/console doctrine:migrations:migrate --no-interaction --env=prod
php bin/console importmap:install --env=prod
php bin/console asset-map:compile --env=prod
php bin/console cache:clear --env=prod
```

Ejemplo de reescritura Nginx: `try_files $uri /index.php$is_args$args;`. La configuración completa del dominio, socket PHP y TLS depende del alojamiento. Para Apache se puede instalar `symfony/apache-pack` y verificar `mod_rewrite` y AllowOverride según el proveedor.

> El torneo es de consulta pública. Las operaciones de registro, edición y eliminación requieren una cuenta administradora. El alojamiento permanente sigue pendiente; el túnel temporal ya se utiliz? para presentar.

Respaldo y recuperación

Respalda MySQL mediante mysqldump --single-transaction y conserva las imágenes junto al archivo SQL. No copies archivos internos de InnoDB mientras está activo. No borres toda la carpeta `var/`: contiene fotos, respaldos y, en esta instancia local, los archivos del servidor MySQL, además de caché y logs. Conserva las copias fuera del directorio web.

Comprobar y presentar el proyecto

```
php bin/phpunit
php bin/console doctrine:schema:validate
php bin/console lint:container
php bin/console lint:twig frontend/templates
```

La suite funcional utiliza SQLite en memoria. Además se ejecutan pruebas de integración sobre MySQL real en una base aislada con otro usuario. Cubre fotografías válidas e inválidas, conservación al editar, errores de escritura, pantallas principales, CRUD de jornadas, validación de goles duplicados, tarjetas y suspensiones, filtros, agregación de goles y protección del historial. También se revisan el menú móvil, la vista previa de imagen y la impresión con un navegador real.

Recorrido sugerido para la exposición

Presenta el inicio; crea un equipo y un jugador; muestra la fotografía; registra una jornada y goles; añade una tarjeta roja; consulta la nómina del árbitro en la fecha de suspensión; filtra incidencias y muestra goleadores; termina explicando entidad → formulario → controlador → Twig. El guion detallado está en `docs/defensa-proyecto.pdf`.

Contenido del paquete

- Código fuente, Composer lock, migraciones, tests y recursos de demostración.
- SQL de datos ficticios, comando de carga y avatares de demostración, sin los registros originales del usuario.
- Manual PDF, fuente HTML editable, README, capturas y reporte PDF de ejemplo.
- Se excluyen credenciales locales, vendor, cachés, logs, herramientas auxiliares y respaldos privados.

Comentarios personales del grupo

Esta sección debe reflejar la experiencia real de los integrantes. No se atribuyen opiniones inventadas. Antes de entregar, respondan brevemente:

- ¿Qué resultó más fácil: las entidades, los formularios, Twig o las migraciones?
- ¿Qué problema concreto encontraron y cómo lo solucionaron?
- ¿Qué aportó cada integrante y qué aprendió?
- ¿Qué cambiarían en una siguiente versión?

Fuentes técnicas

Symfony 7.4: Doctrine y bases de datos
Symfony 7.4: carga de archivos
Symfony 7.4: validación
Symfony 7.4: Callback
Symfony 7.4: publicación
Enunciado proporcionado: ProyectoFinalA.pdf, Universidad Mariano Gálvez de Guatemala.

Conexión y migración de datos

Servidor configurado: MySQL Community Server 8.4.9 en 127.0.0.1:3307. Base: **cantera_torneo**. Usuario: **cantera**. La contraseña está en DATABASE_URL dentro de .env.local. No se incluye en este manual ni en el ZIP.

Se copiaron 5 equipos, 13 jugadores, 4 jornadas, 12 registros de goles y 4 incidencias. Se verificaron todos los campos antes de confirmar la transacción. Las fotografías conservan los mismos nombres y ubicación.

```
php bin/console app:import-sqlite var/torneo.db
```

Este comando ya se ejecutó. No lo repitas sobre una base con datos: se niega a sobrescribirlos. Solo sirve para transferir un respaldo existente a un esquema MySQL vacío.

Explorar la base

En MySQL Workbench utiliza conexión TCP/IP, host 127.0.0.1, puerto 3307 y usuario cantera. Consulta la clave en .env.local. Las tablas incluyen administrador, equipo, jugador, jornada, gol e incidencia; Doctrine añade su tabla de versiones y Messenger su tabla de transporte.

Pruebas y entrega

La cuenta cantera_test solo tiene permisos sobre cantera_torneo_test. Las pruebas revierten sus cambios. El paquete nuevo usa MySQL: los ZIP anteriores eran versiones SQLite y se conservan únicamente como historial.

Consulta la guía completa en `docs/Manual-Cantera.pdf`. Referencia: arranque de MySQL en Windows.

11 / ACCESO

Visitantes y administradores

El enlace del torneo abre el inicio sin pedir una cuenta. Los visitantes pueden consultar el torneo. Para registrar, editar o eliminar, pulsa **Acceso administradores** e inicia sesión. Al terminar usa **Cerrar sesión**.

La cuenta inicial local es admin. Su clave se cambió a petición del usuario el 16/10/2026 y se almacena como hash; var/acceso-admin.txt contiene solo una nota. La clave no se incluye en el manual ni en las entregas. Cada integrante puede tener su propia cuenta:

```
php bin/console app:crear-admin nombre_del_integrante
```

El comando solicita una clave de al menos 12 caracteres y la guarda como hash en MySQL. Si la cuenta existe, permite cambiar su clave tras confirmar. Las cuentas tienen los mismos permisos de administración. No existe registro público de administradores.

La protección se aplica tanto en los controladores como en las rutas; no depende de ocultar botones. El 16/10/2026 pasaron las 8 pruebas de jugadores y fotografías con 217 comprobaciones, incluidas solicitudes directas de creación y edición inválidas y la aceptación al cumplir exactamente 6 años. Se verificaron además el acceso y cierre de sesión en navegador real sobre MySQL y el diseño móvil.

Detalles: docs/Manual-Cantera.pdf.

Arrancar y compartir la página

En cada terminal PowerShell abre `C:\Users\dayli\ProyectoFinalA`. Inicia la base solo si no está ya ejecutándose. XAMPP ocupa el puerto 3306; la instancia del proyecto utiliza el 3307.

Terminal 1 · MySQL

```
& "C:\Program Files\MySQL\MySQL Server 8.4\bin\mysqld.exe" --defaults-
file=C:/Users/dayli/ProyectoFinalA/var/mysql/my.ini
```

Terminal 2 · Aplicación

```
php -S 127.0.0.1:8000 -t public bin/router.php
```

Comprueba primero **http://127.0.0.1:8000**.

Terminal 3 · Enlace público

```
& "C:\Program Files (x86)\cloudflared\cloudflared.exe" tunnel --url http://localhost:8000
```

Copia la URL HTTPS terminada en `trycloudflare.com` que aparece en esa terminal. Puede cambiar al reiniciar el túnel. Mantén la computadora encendida, con Internet, MySQL, PHP y el túnel activos. El túnel no reemplaza un alojamiento permanente. Para terminar usa Ctrl+C en las terminales que iniciaste.

Configuración y cambios posteriores

La copia local para compartir usa `APP_ENV=prod` y `APP_DEBUG=0` en `.env.local`. Después de modificar código o estilos:

```
php bin/console cache:clear --env=prod --no-debug
php bin/console asset-map:compile --env=prod --no-debug
```

No necesitas repetir estos dos comandos en cada presentación. Recarga con Ctrl+F5. Si aparece Access denied para cantera, verifica que DATABASE_URL apunte al puerto 3307. No compartas .env.local ni claves de administración.

Ver los datos en Workbench

Crea una conexión TCP/IP: host `127.0.0.1`, puerto `3307`, usuario `cantera`, esquema `cantera_torneo`. La contraseña está en DATABASE_URL dentro de .env.local. Pulsa Test Connection y abre Schemas → cantera_torneo → Tables; clic derecho en una tabla → Select Rows - Limit 1000.

Revisión técnica del sistema

El 16/10/2026 se ampliaron los controles de seguridad: límite de intentos de login, secreto privado de producción, cookies detrás del proxy, cabeceras HTTP, tokens de eliminación separados, validación compartida de fechas y bitácora por entorno.

Se actualizó Expression Language a 7.4.21 para corregir un aviso de seguridad conocido. La consulta final de Composer Audit no encontró avisos pendientes.

Resultados: 26 pruebas y 475 comprobaciones correctas; 27 comprobaciones de navegador, sin errores JavaScript. Se validaron esquema MySQL, contenedor y 20 plantillas Twig.

La revisión de datos encontró el jugador #14 con nombres, apellidos y nacimiento inválidos. Debe corregirse con datos reales; no se borró. La contraseña débil solicitada para admin y el PHP local antiguo son riesgos pendientes antes de una publicación permanente.

```
php bin/console app:auditar-datos --env=prod --no-debug
```

Este comando no modifica datos. Los informes completos están en **docs/informe-auditoria.pdf** y **docs/informe-funcionamiento.pdf**, con fuentes editables archivadas fuera de docs. La auditoría no garantiza ausencia absoluta de vulnerabilidades ni sustituye una revisión de infraestructura.